# ZOOMER: An APT TTP Recognition System via Deep & Wide Provenance Graph Learning

Xuebo Qiu, Mingqi Lv, Tieming Chen, Tiantian Zhu, *Member, IEEE*, Qijie Song, and Zhiling Zhu

***Abstract*—Advanced Persistent Threats (APTs) commonly manifest through a sequence of attack steps, known as Tactics, Techniques, and Procedures (TTPs). Recent studies identify TTPs by converting audit logs into causal provenance graphs and applying expert-driven mappings that correlate low-level system events with high-level TTP patterns. However, these methods face persistent challenges: determining the impact boundaries of TTP activities, adapting to evolving TTP stacks, and recognizing fine-grained TTP semantics for deeper forensic insights. To address these challenges, we present ZOOMER, a novel TTP recognition framework that segments provenance graphs into multiple TTP subgraphs with multi-granular annotations (i.e., tactics, techniques, and sub-techniques). First, we devise a heuristic subgraph sampling algorithm guided by anomalous node detection to precisely delineate the scope of TTP activities. Second, we introduce a dual-tower Deep & Wide architecture that integrates contextual behavior semantics from provenance graphs and domain-informed features to learn expressive TTP representations. Finally, we adopt a prototypical network that reformulates TTP recognition as a few-shot pattern matching task, thereby enhancing adaptability and accuracy under limited supervision. To advance future research, we built and released the first TTP-annotated provenance dataset, encompassing the most comprehensive collection of TTP instances to date. Extensive experiments show that ZOOMER achieves TTP recognition with 88% accuracy at the sub-technique level and 94% at the tactic level, significantly outperforming state-of-the-art baselines.**

***Index Terms*—Advanced persistent threat (APTs), graph neural network, ATT&CK TTP, data provenance.**

## I. INTRODUCTION

ADVANCED Persistent Threats (APTs) remain one of the most persistent and sophisticated challenges in modern cybersecurity [1]. To inform systematic APT investigation, knowledge bases such as ATT&CK [2] codify the multi-stage lifecycle of APTs into an operational taxonomy of Tactics, Techniques, and Procedures (TTPs), which structures intrusion progression and provides a common vocabulary that significantly supports security research and practice.

In practice, an investigation typically begins when an Endpoint Detection and Response (EDR) system raises an alert [3]. Analysts must then rapidly verify its validity, trace the root cause, and assess its broader impact. Given that alerts in isolation provide insufficient context for multi-stage APT investigation, recent work has utilized system provenance that transforms audit logs into a structured graph (called *provenance graph*) capturing causal relations among system activities [4]. Provenance graphs have empowered attack investigation via graph-theoretic techniques, such as graph compression to alleviate analytical complexity [5], [6], [7], contextual behavior correlation to filter false alarms [8], [9], [10], [11], [12], [13], [14], [15], and automated causal tracing to reconstruct attack scenarios [16], [17], [18], [19], [20], [21].

Despite recent advances, *a fundamental semantic gap persists between low-level provenance data and high-level attack intents characterized by ATT&CK TTPs*, which constrains the practical utility of APT knowledge during investigations. Take the Buran ransomware incident (Fig. 1) [22] as an example. Its lifecycle spans phishing-based initial access (T1566), script execution (T1059), persistence via registry modifications (T1547) and service manipulation (T1543), deletion of recovery mechanisms (T1490), and eventual file encryption (T1486), covering more than ten ATT&CK techniques (detailed in Appendix A, available online). As depicted in Fig. 1(a), these attack semantics (approximately 300 nodes) are deeply buried in a vast provenance graph that consists of more than 40,000 entities and 137,000 events. Investigating attacks in such predominantly benign provenance graphs presents a daunting task. Although intrusion detection systems can reduce the search space by sifting through this background noise to pinpoint suspicious entities [23], [24], [25], [26], [27], their coarse binary classifications (benign or malicious) yield only fragmented anomalies and fail to reveal the potential attack intents. Consequently, analysts are left to rely on their expertise to piece together explanatory views that map observed anomalies to established TTP patterns. For example, as illustrated in Fig. 1(b), the abnormal modification of registry entries by the process `vssvc.exe` can be interpreted as technique T1543. While such semantic correlation yields actionable insights into adversarial intent and significantly aids

Received 22 July 2024; revised 26 September 2025; accepted 15 December 2025. Date of publication 30 December 2025; date of current version 12 May 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U22B2028 and Grant 62372410, in part by the Key Research Program of Hangzhou under Grant 2025SZD1A56, in part by the Zhejiang Provincial Natural Science Foundation of China under Grant LD22F020002, in part by the Key Research Program of Shaoxing under Grant 2025B11004, in part by the Key Research Program of Huzhou under Grant 2025ZD2037, and in part by Zhejiang Province Leading Goose Program under Grant 2025C01013. *(Corresponding author: Mingqi Lv.)*

Xuebo Qiu, Tiantian Zhu, Qijie Song, and Zhiling Zhu are with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou 310023, China, and also with the Zhejiang Key Laboratory of Visual Information Intelligent Processing, Hangzhou 310023, China (e-mail: xueboqiu@zjut.edu.cn; ttzhu@zjut.edu.cn; songqijie@zjut.edu.cn; zhilingzhu@zjut.edu.cn).

Mingqi Lv and Tieming Chen are with the College of Geoinformatics, Zhejiang University of Technology, Huzhou 310014, China, and also with the Zhejiang Key Laboratory of Visual Information Intelligent Processing, Hangzhou 310023, China (e-mail: mingqilv@zjut.edu.cn; tmchen@zjut.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TDSC.2025.3646355, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2025.3646355

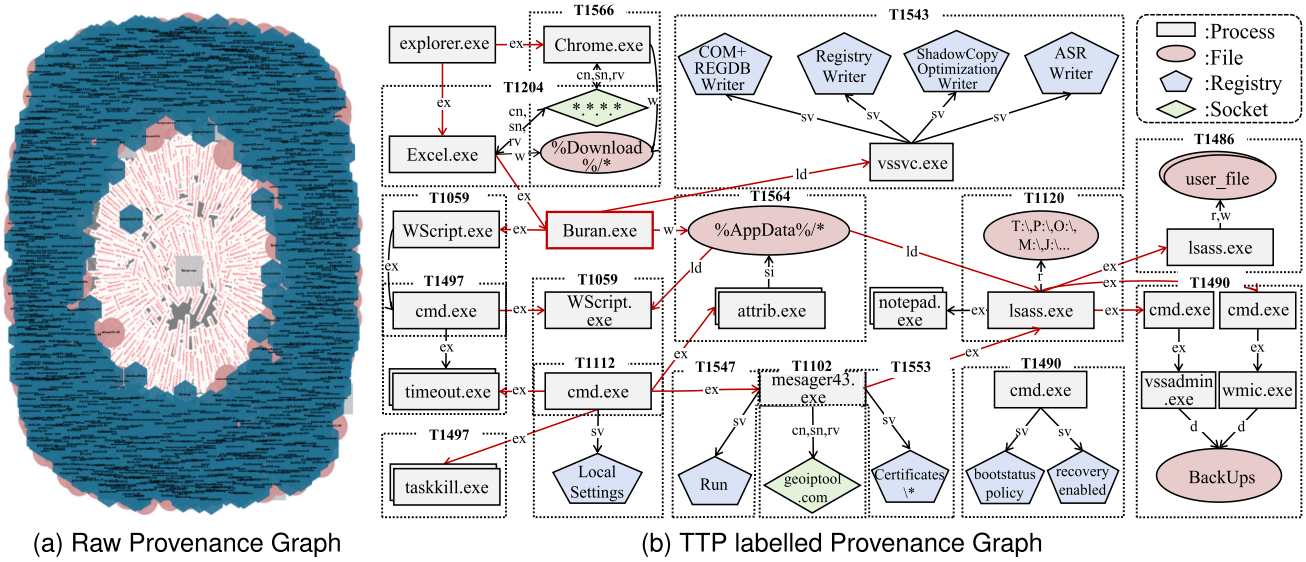


Fig. 1. Provenance graphs of Buran ransomware attack lifecycle: (a) raw graph directly generated from audit logs, (b) simplified graph with TTP labels. Dotted boxes represent different TTP patterns, red edges denote control and data flows between TTPs with operation types (ex=execute, ld=load, r=read, w=write, d=delete, cn=connect, sn=send, rv=receive, sv=set-value, si=set-information), and the red-boxed `Buran.exe` denotes the primary malicious process.

TABLE I
SIGMA RULE FOR T1027 RECOGNITION. ONLY KEY FIELDS ARE RETAINED FOR CLARITY.

**Title:** File Encoded to Base64 via Certutil.exe

**Tags:**
- attack.defense_evasion
- attack.t1027

**Detection:**
selection_img:
- Image|endswith: `\certutil.exe`
- OriginalFileName: `CertUtil.exe`

selection_cli:
- CommandLine|contains|windash: '-encode'

condition: selection_img and selection_cli

investigation (Fig. 1(b)), the process remains labor-intensive, complex and error-prone.

To alleviate these issues, prior research has attempted to link system events to TTPs using expert-crafted rules [6], [16], [28], [29]. For instance, the sigma rule [30] in Table I flags technique T1027 when the process `certutil.exe` is invoked with the "-encode" argument. Unfortunately, the ever-expanding ATT&CK taxonomy (over 531 sub-techniques) renders such heuristic methods difficult to scale and costly to maintain. Moreover, shallow token dependencies (e.g., `certutil.exe` and "-encode" in Table I) make these rules prone to false positives [9], [31]. While follow-up studies enrich rules with multi-step provenance context to reduce false alarms [6], [28], [29], these efforts essentially patch symptoms rather than root causes, resulting in increasingly complex and fragile rule sets. By contrast, deep graph learning offers a promising alternative, as it can autonomously capture high-order dependencies across heterogeneous entities in a data-driven manner, thereby encoding subtle distinctions among TTP patterns. The resulting discriminative representations enable accurate TTP recognition with minimal human involvement. Nevertheless, training fine-grained TTP recognition models over large-scale provenance graphs introduces the following unique challenges, which this work aims to address.

*C1. Ambiguous Recognition Targets:* Unlike APT detection, which has well-defined targets (i.e., a specific node or graph [23], [24]), TTP recognition often involves indistinct boundaries. Within a provenance graph, the scope of a TTP instance is difficult to delineate, as it may span from only a few nodes to several hundred. This ambiguity complicates the definition of recognition units.

*C2. Expansive and Evolving Label Space:* TTP recognition is inherently a multi-class classification task with an unusually large label space. The ATT&CK matrix defines over 531 enterprise techniques and sub-techniques, and this stack continually expands as new TTPs are introduced. Such scale and dynamism strain model accuracy and adaptability.

*C3. Complexity and Diversity of TTP Features:* Although existing provenance graph representations have proven effective in differentiating benign from malicious activities, they struggle to capture discriminative features across the diverse spectrum of TTP patterns. This limitation arises because distinctions between TTPs are often subtle (e.g., confusing T1003.001-9 with T1222.001-4, as discussed in Section IV-D), making accurate recognition particularly difficult.

*C4. Scarcity of TTP Samples:* The sophistication and prolonged duration of APT campaigns lead to limited availability of TTP samples. Most public datasets are curated for APT detection rather than TTP recognition, and thus provide restricted TTP coverage. For example, DARPA TC E3 [32] includes merely 27 TTPs and lacks annotations. This scarcity hinders effective model training and generalization.

To address these challenges, we introduce ZOOMER[1], a learning-based TTP recognition framework that segments provenance graphs into TTP-specific subgraphs and assigns multi-granular labels (i.e., tactics, techniques, and sub-techniques) with both efficiency and accuracy. To tackle challenge C1, a heuristic subgraph sampling algorithm that builds on the detection of anomalous process nodes, referred to as Nodes of Interest (NOIs), is proposed to identify the boundaries of TTP instances. For challenge C3, ZOOMER employs a dual-tower Deep & Wide architecture [33], where a graph learning-based model serves as the deep component to extract structural and semantic behavior features for broad generalization, while a knowledgeguided generalized linear model serves as the wide component to memorize domainspecific TTP features. By integrating these complementary features, ZOOMER constructs highly discriminative TTP representations. To address challenges C2 and C4, ZOOMER adopts a few-shot learning method based on the prototypical network, and transforms TTP recognition from a multi-class classification task into a metric-based pattern matching problem. To support rigorous evaluations, we further built and released the KELLECT4APT dataset [34] by executing atomic TTP scripts from Red Canary[2] [35] and collecting the generated audit logs with KELLECT[3] [36]. The dataset spans 9 tactics, 26 techniques, 45 sub-techniques, and 473 procedures, representing the most comprehensive TTP corpus to date. Our contributions are summarized as follows:

- We propose a knowledge- and data-driven framework capable of performing TTP recognition at multiple granularities based on prototypical network.
- We design an NOI detection model combined with a heuristic subgraph sampling algorithm to efficiently segment TTP subgraphs from provenance graphs.
- We introduce a Deep & Wide architecture that extracts generalizable structural and semantic behavior features, and memorizes domain-informed features, thereby yielding robust and discriminative TTP representations.
- We build and release the first provenance dataset with comprehensive TTP annotations, establishing a solid benchmark for future research.
- We conduct extensive experiments that demonstrated ZOOMER's effectiveness, achieving recognition of 160 TTPs with 88% accuracy at the sub-technique level and 94% accuracy at the tactic level.

## II. RELATED WORK AND LIMITATIONS

### A. APT Detection

Current APT detection studies generally fall into rule-based and learning-based paradigms. Rule-based methods encode expert knowledge into handcrafted detection policies [6], [18], [19], [28], [29], [37], [38], [39], [40]. For example, Holmes [28] and Rapsheet [31] identify suspicious behaviors via single-hop pattern matching, while Tags [37] and Sleuth [39] incorporate contextual interactions to improve detection robustness and reduce false alarms. Although computationally efficient, these methods often fail to generalize to unseen attacks. Learning-based methods instead employ deep learning models trained on large-scale provenance data to detect intrusions. Early supervised learning approaches [11], [18], [19], [26], [27], [41] train models on labeled attack samples using specialized optimization mechanisms. However, given the scarcity of labeled data, recent work [23], [24], [42], [43], [44], [45], [46] has shifted to unsupervised learning paradigms. These methods model benign behaviors on provenance data at scale and flag deviations as potential APTs [23], [24], [42], [43], [44], [45], [46], or cluster malicious network events to identify coordinated campaigns [47]. Recent provenance-based detectors further exploit Graph Neural Networks (GNNs) with self-supervised learning to capture normal system behaviors. For instance, MAGIC [24] employs a graph masked auto-encoder to reconstruct masked node features and structures, thereby encoding high-order semantic and structural patterns of benign activity. Flash [25] integrates a word semantic encoder with a GNN-based contextual encoder to generate semantically rich embeddings of benign nodes. THREATRACE [23] adapts an inductive GNN to learn benign entity roles in streaming provenance data for real-time anomaly detection. The trained model flags anomalies as deviations from the distribution of learned benign representations. Despite their effectiveness, these methods remain limited to flagging isolated anomalies (e.g., individual nodes or edges in Fig. 1(b)) without clarifying how they map to higher-level TTP patterns or delineating their boundaries. These limitations motivate our work on TTP recognition, which explicitly extracts TTP subgraphs and infers their underlying patterns, thereby supporting more structured and insightful attack investigations.

### B. TTP Recognition.

TTP recognition aims to align anomalous system behaviors with TTP patterns, thereby enriching the semantic context of attack investigations. Prior methods predominantly rely on expert-crafted mappings. For instance, Holmes [28] formulates multiple TTP specifications to associate events with Kill-Chain tactics, while E-Audit [18] correlates EDR rules with ATT&CK techniques through statistical associations. However, their limited use of contextual anomaly information often causes elevated false positives, particularly when benign processes mimic TTP-like behaviors. For instance, routine scheduled scripts executed via `Powershell` may be mistakenly classified as the "T1059: Command and Script Interpreter" technique [48]. Later studies address this by incorporating richer execution context [6], [18], [28], [29]. APTShield [6], for instance, propagates suspicious states across system entities using curated transition rules and black/white lists (e.g. sensitive files) to infer tactic-level predictions. Similarly, Conan [29] models contextual behaviors with a with a Finite State Automata (FSA) framework to capture state transitions. Despite these advances, such methods remain tied to handcrafted rules, which hinders scalability and generalization. More critically, static rule design confines recognition to

[1]The system is named ZOOMER for its ability to correlate low-level system events with TTP patterns at multiple granularities (i.e., tactics, techniques, and sub-techniques), similar to the zooming effect of a lens.

[2]Red Canary is an industry-leading threat detection and response provider.

[3]KELLECT is our self-developed tool for collecting system audit events.

TABLE II
MONITORED SYSTEM ENTITIES AND EVENTS

| System Entities | System Events |
|---|---|
| Process → Process | start, end |
| Process → File | iocreate, iodirenum, ioread, iofilecreate, iowrite, iodelete, iofiledelete, iodirnotify, iorename |
| Process → Registry | open, queryvalue, query, create, setvalue, setinformation, enumeratevaluekey, enumeratekey, kcbcreate, querymultiplevalue, deletevalue, delete, kcbdelete, flush |
| Process → Socket | sendipv4, connectipv4, recvipv4, disconnectipv4, disconnectipv6, reconnectipv4, reconnectipv6, acceptipv4, tcpcopyipv4, connectipv6, sendipv6, recvipv6 |

coarse-grained tactics and fails to delineate the precise boundaries of fine-grained behaviors. These limitations motivate our design of a generalized TTP recognition system that simultaneously extracts TTP behavior subgraphs and identifies fine-grained patterns (e.g., techniques and sub-techniques), thereby offering deeper insights to strengthen APT investigations.

### C. Threat Hunting.

Recent advances in threat hunting [7], [49], [50], [51] adopt graph pattern matching to align provenance graphs with attack campaigns documented in Cyber Threat Intelligence (CTI) reports. For instance, Poirot [49] employs heuristic queries to retrieve abnormal system activities resembling CTI-recorded attacks. More recently, GNNs have been introduced to model complex behavior semantics in provenance data. Based on this, GHunter [51] learns hierarchical behavior representations via order embeddings [52] to predict approximate subgraph relationships between CTI-derived attack graphs and provenance graphs, while ProvG-Searcher [7] mitigates dependency explosion and GNN over-smoothing through graph partitioning and compression, followed by process-centric subgraph entailment prediction. Although these methods advance embedding-based matching for APT hunting, they largely focus on coarse pattern similarities to confirm attack presence, while struggle to distinguish subtle variations across TTP patterns. This limitation motivates us to develop a more expressive TTP representation model that captures fine-grained distinctions among TTPs, thereby enabling more precise recognition of TTP subgraphs and enhancing the fidelity of APT investigations.

## III. BACKGROUND

### A. Definitions

*Provenance Graph:* A provenance graph (Fig. 1(b)) organizes system entities and their interactions into a graph structure [4]. Formally, it is defined as $P = (V, E)$, where $V$ denotes entities (e.g., process, file) and $E$ denotes their interactions (e.g., read, write). An event $e = \{u, v, a\} \in E$ connects two system entities $u, v \in V$ through an interaction $a$. Table II summarizes the monitored entities and events.

*TTP Subgraph:* The ATT&CK framework [2] hierarchically abstracts adversarial behaviors into Tactics, Techniques, and Procedures (TTPs). Tactics denote high-level adversarial objectives, techniques describe the means to achieve these objectives, sub-techniques further refine them into more granular actions, and procedures represent concrete implementations. We define a TTP subgraph ($TSG$) as a provenance graph segment corresponding to a procedure under a specific technique or sub-technique, presented by the dashed boxes in Fig. 1(b).

*Deep & Wide Architecture:* The Deep & Wide architecture enhances user preference prediction in recommender systems [33]. It combines a wide linear model that captures explicit feature interactions to memorize user preferences, and a deep neural network that extracts dense embeddings to support generalization. Motivated by the parallels between its design philosophy and TTP recognition objectives, we adapt this architecture to jointly learn knowledge-driven and data-driven TTP representations.

*Prototypical Network:* The prototypical network [53] is a few-shot learning framework that learns a discriminative metric space, where classification can be performed by measuring the distance between query samples and class prototypes. This approach enables effective generalization from limited labeled data, making it well-suited for TTP recognition, where annotated samples are inherently scarce.

### B. Problem Statement

This work focuses on delineating TTP behavior boundaries in provenance graphs and identifying their multi-granular patterns. Concretely, given a real-time stream of audit events that comprise numerous benign behaviors and a small fraction of malicious ones, our goal is to precisely extract the involved *TSGs* and assign them ATT&CK TTP labels, thereby enriching semantic insights of the attack investigation beyond binary malicious predictions. Importantly, our work differs from APT attribution [54], [55], which seeks to identify the specific threat actor (e.g., a known APT group) behind an attack. Nevertheless, the enriched TTP semantics produced by ZOOMER could serve as a valuable input to attribution pipelines.

### C. Threat Model

We consider the auditing framework, operating systems, and system logs are trustworthy, meaning that the observed system activities are not tampered with. Adversaries are presumed to conduct APT campaigns by leveraging diverse tactics and techniques defined in ATT&CK. However, threats that cannot be captured by the auditing framework, such as side-channel attacks, kernel-level exploits, or hardware-based attacks, are considered out of scope. Moreover, we assume that TTPs belonging to the same technique or sub-technique category share consistent contextual semantics, while TTPs across different categories exhibit distinguishable patterns.

## IV. SYSTEM DESIGN

### A. Overview

The architecture of ZOOMER is presented in Fig. 2, which comprises five modules: 1) provenance graph construction,

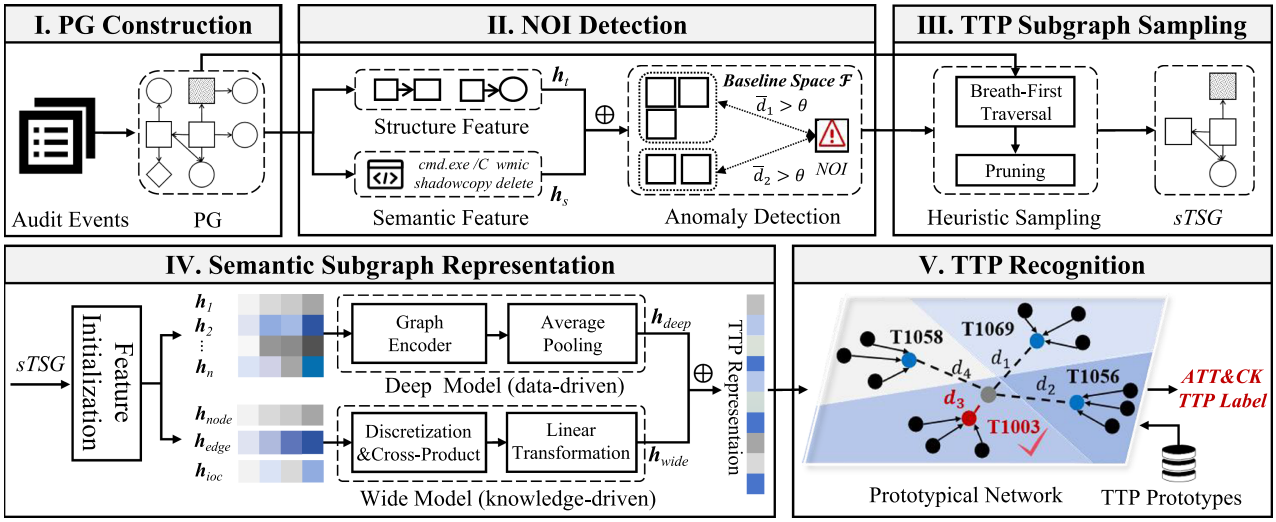


Fig. 2. Overview of ZOOMER architecture. PG: Provenance Graph. $\oplus$: Feature Concatenation.

2) NOI detection, 3) TTP subgraph sampling, 4) semantic subgraph representation, and 5) TTP recognition. Initially, streaming audit logs are iteratively parsed to extract relevant entities and their interactions. These extracted elements are subsequently transformed into provenance graphs, where various compression strategies [5], [6] are applied to remove redundant interactions. Following the graph construction, the NOI detection model locates anomalous process nodes within the provenance graph. Guided by these NOIs, a heuristic subgraph sampling algorithm extracts candidate $TSGs$ centered around them. After that, the semantic subgraph representation model takes each $TSG$, derives both data-driven and knowledge-driven features, and projects them into a dense embedding space. Finally, a prototypical network tailored for few-shot graph learning compares the $TSG$ embeddings against TTP prototypes to achieve multi-granular TTP recognition. We elaborate these designs in the following subsections.

### B. NOI Detection

APT campaigns are camouflaged within a sea of benign activities, making exhaustive search and analysis of TTP patterns computationally prohibitive (challenge C1). As TTP activities are typically initiated around anomalous process nodes [7], [27], [56], we designate process NOIs as primary analysis targets. Focusing on these nodes could substantially narrow down the search space and improve the efficiency. Furthermore, given the high cost of manually annotating malicious samples for supervised training, we adopt an unsupervised anomaly detection approach, whose robustness under limited supervision makes it particularly suitable for our task. Concretely, we first model a baseline distribution of normal process behaviors from a large corpus of attack-free provenance graphs, and then flag a process node as an NOI if its behaviors significantly deviate from this baseline. The remainder of this section details node feature initialization and anomaly detection methods.

*1) Feature Initialization:* For each process node, we extract two feature types to characterize its behaviors.

*Structural Feature:* Malicious nodes typically display interaction patterns with their neighbors that differ from those of benign nodes, resulting in distinct local graph structures [23]. Based on this insight, we consider 37 event types and form a 74-dimensional structural feature $\boldsymbol{h}_t = [a_0, a_1, a_2, \ldots, a_{36}, \ldots a_{73}]$ for each process, where the first 37 dimensions $a_i (0 \leq i \leq 36)$ represent counts of incoming edges by event type, while the second 37 dimensions $a_j (37 \leq j \leq 73)$ correspond to counts of outgoing edges.

*Semantic Feature:* Leveraging structural features alone for NOI detection can result in high false positives, as benign and malicious processes may share similar local structures. For instance, both legitimate system backup processes and malicious data exfiltration processes may read files and transmit data to a cloud server. Generally, the command line of a process reflects its functionality, which motivates us to incorporate such semantics as complementary features for NOI detection. To this end, we employ a hierarchical hashing scheme [42], [57] to encode command lines. First, each command line is decomposed into hierarchical substrings. For example, given a command line "C:\Windows\regedit.exe", we generate its hierarchical substrings $s_1$ = "C:", $s_2$ = "C:\Windows" and $s_3$ = "C:\Windows\regedit.exe". Each substring is then embedded via hashing: $\phi_i(s) = \sum_{k:h(s_j)=i} \mathcal{H}(s_i)$, where $\phi_i$ represents the $i$-th dimension of the embedding, $s_j$ denotes the $j$-th character in $s$, $h(\cdot)$ maps $s_j$ to a dimension index, and $\mathcal{H}(\cdot)$ is another hash function mapping $s_j$ to $\{\pm 1\}$. $\mathcal{H}(s_j)$ would be added to the $i$-th dimension if $h(s_j) = i$. Finally, substring embeddings are aggregated to yield a 32-dimensional semantic feature vector $\boldsymbol{h}_s$. Note that the encoding scheme can be replaced by any methods that capture text similarity [25], [27].

After initialization, we normalize $\boldsymbol{h}_t$ and $\boldsymbol{h}_s$ to mitigate scale discrepancies, and then concatenate them into a unified feature vector to represent each process node.

*2) Anomaly Detection:* We employ a $k$-nearest neighbors (KNN) detector [58] for distance-based anomaly detection. The baseline feature space $\mathcal{F}$ is constructed from feature vectors of process nodes in benign provenance graphs, representing the

**Algorithm 1:** NOI Detection.

**Input:** Provenance graph $G = (V, E)$, baseline feature space $\mathcal{F}$, anomaly threshold $\theta$, neighborhood size $k$
**Output:** Set of process NOIs $N$

1 $N \leftarrow \emptyset$;
2 $\boldsymbol{X} = FeatureInitialization(G)$;
3 **foreach** *process node* $p \in V$ **do**
4 $\quad \mathcal{N}_k \leftarrow KNN(\boldsymbol{X}[p], \mathcal{F}, k)$;
5 $\quad \bar{d} \leftarrow \frac{1}{k} \sum_{\boldsymbol{h}' \in \mathcal{N}_k} ||\boldsymbol{X}[p] - \boldsymbol{h}'||_2$;
6 $\quad$ **if** $\bar{d} > \theta$ **then**
7 $\quad\quad N \leftarrow N \cup \{p\}$; // mark as NOI
8 **return** $N$

**Algorithm 2:** TTP Subgraph Sampling.

**Input:** Provenance graph $G = (V, E)$, process NOI set $N$
**Output:** Set of TTP subgraphs $TSGs$

1 $S_{TSG} \leftarrow \emptyset$; // Initialize the set of $TSGs$
2 **foreach** $noi \in N$ **do**
3 $\quad queue \leftarrow \{noi : (+\infty, -\infty, 0)\}$, $sg \leftarrow \emptyset$;
4 $\quad$ **while** $queue \neq \emptyset$ **do**
5 $\quad\quad node, (t_{in}, t_{out}, depth) \leftarrow queue.pop()$;
6 $\quad\quad$ **if** $depth > 2$ **then**
7 $\quad\quad\quad$ **continue**;
$\quad\quad$ /* Incoming Edge Processing */
8 $\quad\quad$ **foreach** $(u, node, time) \in E$ **do**
9 $\quad\quad\quad$ **if** $u \notin queue$ ***and*** $time < t_{in}$ **then**
10 $\quad\quad\quad\quad queue[u] \leftarrow (time, +\infty, depth + 1)$;
11 $\quad\quad\quad\quad sg \leftarrow sg \cup \{(u, node, time)\}$;
$\quad\quad$ /* Outgoing Edge Processing */
12 $\quad\quad$ **foreach** $(node, v, time) \in E$ **do**
13 $\quad\quad\quad$ **if** $v \notin queue$ ***and*** $time > t_{out}$ **then**
14 $\quad\quad\quad\quad queue[v] \leftarrow (-\infty, time, depth + 1)$;
15 $\quad\quad\quad\quad sg \leftarrow sg \cup \{(node, v, time)\}$;
16 $\quad S_{TSG} \leftarrow S_{TSG} \cup \{TransformSubgraph(sg)\}$;
17 **return** $S_{TSG}$

**Algorithm 3:** TTP Subgraph Pruning.

**Input:** Set of TTP subgraphs $S_{TSG}$
**Output:** Set of pruned TTP subgraphs $S_{TSG}$

1 **foreach** $sg \in S_{TSG}$ **do**
$\quad$ /* Remove dependency explosion nodes */
2 $\quad$ **foreach** $v \in sg.V$ **do**
3 $\quad\quad$ **if** $isHighDegree(sg, v)$ **then**
4 $\quad\quad\quad sg.V \leftarrow sg.V \setminus \{v\}$;
5 $\quad\quad\quad sg.E \leftarrow sg.E \setminus \{e | e.src = v \vee e.dst = v\}$;
$\quad$ /* Apply heuristic filtering */
6 $\quad sg \leftarrow FilterFrequentEvents(sg)$;
7 $\quad sg \leftarrow RemoveSingularNodes(sg)$;
8 $\quad sg \leftarrow FilterUninformativeEvents(sg)$;
9 $S_{TSG} \leftarrow MergeSubgraphs(S_{TSG})$;
10 **return** $S_{TSG}$

distribution of normal behaviors. During inference, as outlined in Algorithm 1, each provenance graph is first initialized to obtain the feature matrix $\boldsymbol{X}$ (line 2). For each process node, its $k$ nearest neighbors are then retrieved from $\mathcal{F}$ (lines 3–4), and the average Euclidean distance $\bar{d}$ to these neighbors is computed (line 5). If $\bar{d}$ exceeds a predefined threshold $\theta$, the process is flagged as anomalous (lines 6-7). The detected NOIs subsequently serve as anchors for extracting $TSGs$.

### C. TTP Subgraph Sampling

Precisely delineating the impact scope of a $TSG$ is crucial for effective TTP recognition and attack investigation (challenge C1). To this end, we propose a heuristic sampling algorithm that segments provenance graphs into individual $TSGs$, which then serve as the recognition units. Empirical analysis on the ATT&CK framework reveals that each $TSG$ features a process node as the principal executor, with auxiliary nodes supporting the tactic objective. Guided by this observation, our sampling algorithm anchors on process NOIs and iteratively expands along neighboring nodes to capture semantically relevant context.

*1) Suspicious Subgraph Sampling:* Prior sampling methods often perform forward-only traversal to extract behavior instances [59], which inadvertently disregards preceding causal dependencies essential for comprehensive behavior representation. To overcome this limitation, we propose a temporal-aware breadth-first traversal that samples both backward (incoming) and forward (outgoing) dependencies while strictly following event timestamps. The full procedure is outlined in Algorithm 2. For each process NOI (line 2), incoming edges are explored in reverse chronological order (lines 8–11) and outgoing edges are sampled in chronological order (lines 12–15). To control graph expansion and sampling cost, we impose a 2-hop traversal limit (lines 6–7), an empirically supported bound of $TSGs$ in KELLECT4APT.

*2) Subgraph Pruning:* After the initial subgraph sampling, two major issues arise that undermine the semantics of $TSGs$. First, dependency explosion nodes (e.g., long-running processes or shared system files) introduce large volumes of benign interactions that obscure TTP activities. Second, the sampled $TSGs$ encapsulate edges that are part of routine tasks, such as `powershell` loading system dynamic link libraries (detailed in Appendix B, available online). To address these issues, we develop a subgraph pruning algorithm, as outlined in Algorithm 3. We begin by pruning high-degree nodes (i.e., potential dependency explosion nodes [59], [60]) along with their edges (lines 2–5). Then, we apply three heuristics to further eliminate noise (lines 6–8): 1) filter out edges with historically high occurrence frequency; 2) remove file and registry nodes singly connected to one process; and 3) discard low-informative edge types (e.g., open, close). The first rule primarily eliminates routine system or user activities characterized by high frequency, whereas the latter two suppress "static" nodes and edges, such as those involving log or temporary files [7], [59], which contribute little to meaningful TTP representation. Finally, pruned $TSGs$ with overlapping nodes are merged (line 9) to preserve the coherence of anomalous behaviors while still maintaining clear boundaries across distinct TTP instances. The resulting disjoint subgraphs serve as the final targets for TTP recognition.

### D. Semantic Subgraph Representation

TTP activities are inherently complex and diverse (challenge C3). Distinct TTPs may induce subgraphs with similar structures but divergent intents, while the same TTP can also appear in structurally different forms. For example, as depicted in Fig. 3(a)

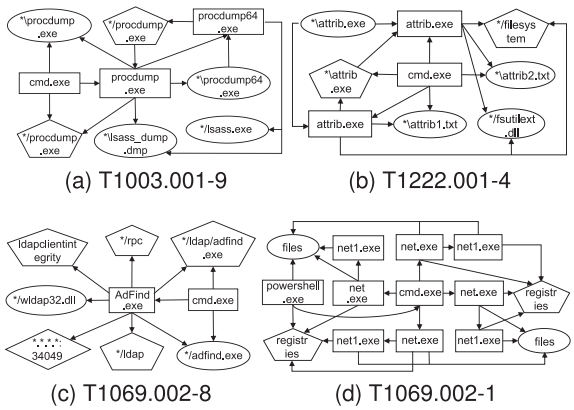


Fig. 3. Structural comparison of different TTPs.

and (b), T1003.001-9 leverages the `procdump` utility to dump ttlsass.exe memory for credential theft, whereas T1222.001-4 utilizes the `attrib` program to conceal system files. Although their objectives differ, these procedures produce structurally similar subgraphs, which can lead to misclassification. Conversely, both T1069.002-8 and T1069.002-1 pursue domain group enumeration. However, as revealed in Figs. 3(c) and (d), T1069.002-8 leverages `Adfind` for active directory reconnaissance, whereas T1069.002-1 relies on `net` for permission group queries, resulting in markedly different subgraph structures. Collectively, such findings reveal the inadequacy of current purely structural encoding [23], [26], [51], and point to the necessity of incorporating richer semantics into TTP representations.

In response, we introduce a dual-tower Deep & Wide architecture that integrates data-driven feature learning with knowledge-driven pattern memorization to capture richer semantics. This architecture, originally proposed for recommender systems [33], aligns with our task because both domains share the underlying objective of balancing generalization and memorization. In recommendation, the deep model generalizes by learning latent user preference patterns, whereas the wide model memorizes frequent user-item interactions for improved accuracy. Similarly, ZOOMER leverages the deep model to learn generalized representations of diverse TTP activities from semantic and structural features, while the wide model memorizes explicit feature combinations (e.g., IoCs) tied to specific TTPs. These two complementary mechanisms yield expressive and robust TTP representations.

*1) Deep Model:* The deep model functions as a graph encoder to autonomously extract intricate interaction patterns among system entities. Specifically, we employ the Graph Attention Network (GAT) [61], which assigns adaptive attention weights to neighbors rather than treating them uniformly as in conventional GNNs. This enables the model to emphasize informative interactions while suppressing trivial ones, a property that is valuable for provenance graphs, where critical behaviors (e.g., sensitive file accesses) are often buried within abundant benign context (e.g., temporary file reads).

With this motivation, we initialize node features using the structural and semantic features described in Section IV-B, where semantic features are extracted from file paths for file nodes, registry keys for registry nodes, IP addresses for socket nodes, and command lines for process nodes. These features only capture first-order interactions and fail to model higher-order contextual dependencies, which are crucial for accurately representing system behaviors. To address this, we stack multiple GAT layers to recursively incorporate information from multi-hop neighborhoods and thus capture richer contextual interactions. Through this design, the representation of node $i$ is iteratively updated through weighted aggregation of its neighbors across multiple layers. Formally, at the $k$-th layer, the attention weight $e_{ij}$ between nodes $i$ and $j$ is derived from their semantic similarities:

$$e_{ij} = \sigma(\boldsymbol{W}_a(\boldsymbol{h}_i^{t-1} \oplus \boldsymbol{h}_j^{t-1})), \tag{1}$$

where $\boldsymbol{h}_i^{t-1}$ and $\boldsymbol{h}_j^{t-1}$ denote the features of nodes $i$ and $j$ at the $(t-1)$-th layer, $\oplus$ stands for the concatenation operation, $\boldsymbol{W}_a$ is a learnable parameter matrix, and $\sigma$ is a nonlinear activation function (e.g., ReLU [62]). To guarantee non-negativity of the attention weights, a softmax normalization is applied over all neighborhood interactions:

$$\alpha_{ij} = \frac{exp(e_{ij})}{\sum_{k \in N_i} exp(e_{ik})}, \tag{2}$$

where $N_i$ is the neighbor set of node $i$. To enhance expressiveness, we incorporate a multi-head attention mechanism that performs $K$ parallel attention computations, allowing the model to attend to complementary interaction semantics across distinct subspaces. The representation of node $i$ is then computed as a nonlinear transformation of the weighted aggregation of its neighbors' embeddings:

$$\boldsymbol{h}_i^t = \oplus_{k=1}^{K} \sigma\left(\boldsymbol{W}^k \boldsymbol{h}_i^{t-1} + \sum_{v_j \in N_i} \alpha_{ij}^k \boldsymbol{h}_j^{t-1}\right), \tag{3}$$

where $a_{ij}^k$ is the attention coefficient from the $k$-th head, and $\boldsymbol{W}^k$ is its learnable parameter matrix. After stacking $t$-layer GATs, we obtain the final set of node embeddings $\boldsymbol{X}$.

The deep representation $\boldsymbol{h}_{deep}$ of each $TSG$ is finally obtained by aggregating all node embeddings via average pooling:

$$\boldsymbol{h}_{deep} = \frac{1}{N} \sum_{i=1}^{N} \boldsymbol{X}[i]. \tag{4}$$

*2) Wide Model:* The wide model is implemented as a generalized linear model designed to capture explicit feature interactions. It complements the deep model by memorizing knowledge-driven features that exhibit strong correlations with specific TTPs. To this end, we design three heuristic feature families, namely node characteristics, edge relations, and IoCs distributions, which together offer a complementary perspective on TTP semantics.

*Abstract Node Feature:* Nodes are abstracted into semantic types based on their intrinsic attributes (Table III). For example, a file node "C:\Windows\System32\kernel32.dll" can be abstracted into types "system_file" and "dll_file". Such abstraction enables the capture of semantic similarities in entity distributions across various TTPs, thereby strengthening the wide model's

TABLE III
ABSTRACT NODE TYPES

| Node Type | Abstract Node Type |
|---|---|
| Process | rootProcess, systemProcess,userProcess |
| File | systemFile, tmpFile, logFile, exeFile, dllFile,scriptFile, dirFile, otherFile |
| Registry | HKLM, HKCU, HKCR, HKU,otherReg |
| Socket | internalIP,externalIP |

capacity to memorize recurrent patterns. The resulting feature vector $\boldsymbol{h}_{node}$ is 18-dimensional.

*Edge Feature:* The distribution of edge types within a $TSG$ reflects statistical evidence of behavior signatures. For instance, "T1485: Data Destruction" that aims to destroy system and user files typically manifests frequent file operations. By capturing such regularities, edge feature allows the wide model to memorize shared interaction characteristics among different TTPs. Following Table II, we construct a 37-dimensional feature vector $\boldsymbol{h}_{edge}$ encapsulating these statistics.

*IoC Feature:* To enhance interpretability and robustness, we incorporate IoC features contextualized by ATT&CK tactics. Such domain features are initialized through knowledge bases derived from established CTI repositories. Since most CTI repositories (e.g., Red Canary) describe unstructured associations between IoCs and APT tactics, we design an automated extraction framework to leverage this knowledge. For unstructured CTI sources, we apply regex-based matching to identify IoCs and their associated tactics [2]. For structured sources, we directly parse formatted fields (e.g., YAML entries). Examples of the extracted mappings are presented in Appendix C, available online. The resulting IoCs span file paths, registry keys, and C&C domains, covering 11 ATT&CK tactics. Each $TSG$ is thus represented as a 33-dimensional feature vector $\boldsymbol{h}_{ioc}$, where each dimension encodes the occurrence frequency of IoCs associated with a specific tactic.

These domain-informed features are continuous values spanning broad and disparate ranges. Learning stable interaction patterns from these variable numerical features poses a significant challenge [63]. To this end, we propose two feature optimization strategies to improve their utility. First, a feature discretization mechanism [64] is employed to transform continuous values into categorical formats, thereby emphasizing recurring patterns and reducing variability in raw numeric inputs. This transformation is achieved using a clustering-based method [65], where each feature dimension is partitioned into a set of disjoint intervals (e.g., [0,20], [21,34], [35,46], and [47,60] for the "system_file" dimension). Continuous values are then mapped to one-hot vectors corresponding to their respective intervals (e.g., a value of 24 is encoded as [0,1,0,0]). The resulting categorical features $\boldsymbol{h}_{cat}$ allow the model to more effectively capture common patterns. Second, we apply a Cross-Product transformation to $\boldsymbol{h}_{cat}$ to explicitly generate higher-order feature interactions that facilitate model learning. Formally, a single transformation is defined as:

$$\Phi(\boldsymbol{h}) = \prod_{i=1}^{d} \boldsymbol{h}_i^{c_i}, \quad c_i \in \{0, 1\}, \tag{5}$$

where $c_i$ is a randomly sampled binary indicator that determines whether the $i$-th feature is included (1) or excluded (0), and $d$ is the feature dimensionality. By applying this transformation $K$ times over $\boldsymbol{h}_{cat}$ with different random selections, we produce a diverse set of cross-product features. Finally, these generated features are concatenated with the original $\boldsymbol{h}_{cat}$ and projected through a linear layer to form a 64-dimensional representation $\boldsymbol{h}_{wide}$ for each $TSG$:

$$\boldsymbol{h}_{wide} = \boldsymbol{W}_w[\boldsymbol{h}_{cat} \oplus \Phi_1(\boldsymbol{h}_{cat}) \oplus \cdots \oplus \Phi_K(\boldsymbol{h}_{cat})] + \mathbf{b}_w, \tag{6}$$

where $\boldsymbol{W}_w$ and $\mathbf{b}_w$ are the learnable parameters. Finally, the representation of each $TSG$ is constructed by concatenating $\boldsymbol{h}_{wide}$ with the deep representation $\boldsymbol{h}_{deep}$.

### E. TTP Recognition

A simple solution for TTP recognition is to train a multi-class classifier that distinguishes TTP patterns by learning class-specific decision boundaries. However, the scarcity of labeled data and the ever-expanding TTP stack make this strategy neither practical nor scalable (challenges C2 and C4). Consequently, we adopt the prototypical network [53], a representative metric-based few-shot learning framework that transforms TTP recognition from a classification problem into a pattern matching problem. Within this paradigm, each TTP is represented by a prototype vector, and a discriminative metric space is learned to enforce samples to cluster around their respective prototypes while remaining well separated from others. This approach not only allows TTP recognition with a handful of labeled samples but also readily accommodates the emergence of novel TTPs.

*1) Model Training:* The prototypical network is trained in a $N$-way $K$-shot setting, where $N$ denotes the number of classes and $K$ denotes the number of samples per class. Training episodes are constructed from the KELLECT4APT dataset. Each episode consists of a support set covering 32 TTP classes with 3 samples per class (32-way 3-shot), while the query set is formed from the remaining 2-3 samples per class. The prototype vector $\boldsymbol{c}_k$ for the $k$-th TTP class is derived by averaging the embeddings of its support samples $S_k$:

$$\boldsymbol{c}_k = \frac{1}{|S_k|} \sum_{x \in S_k} f_\phi(x), \tag{7}$$

where $f_\phi$ is our subgraph representation module. For each query sample $x$, we calculate its Euclidean distance to each prototype vector as $d(x, \boldsymbol{c}_k) = \|f_\phi(x) - \boldsymbol{c}_k\|_2$. The cross-entropy loss is utilized to minimize intra-class distances while maximizing inter-class separation for model optimization.

*2) Model Inference:* During inference, a sampled $TSG$ is provided as input $x$ and embedded via the subgraph representation module $f_\phi$. The embedding is then compared against all TTP prototypes $\{\boldsymbol{c}_k\}_{k \in \mathcal{C}}$ in the learned metric space, and the predicted class is determined based on the nearest prototype under Euclidean distance:

$$k(x) = \arg\min_{k \in \mathcal{C}} \|f_\phi(x) - \boldsymbol{c}_k\|_2, \tag{8}$$

where $k(x)$ denotes the predicted label of $x$. To reduce false positives, a decision constraint is imposed: if the minimum distance to any prototype exceeds a predefined threshold, the sample is classified as benign.

## V. Experiment

This section presents a comprehensive experimental evaluation of ZOOMER. Section V-A introduces the experimental setup, including dataset descriptions and evaluation strategies. In Section V-B, we conduct parameter sensitivity analysis on different components of ZOOMER to determine the optimal configuration. Section V-C provides ablation studies analyzing the importance of the deep and wide models in the TTP representation module. We then compare ZOOMER against baseline methods for both NOI detection and TTP recognition in Section V-D. Section V-F evaluates its computational overhead. Section V-G demonstrates ZOOMER's effectiveness on real-world datasets. Finally, Section V-H presents a case study that qualitatively illustrates how ZOOMER facilitates attack investigation in practice.

### A. Experiment Setup

*1) Dataset:* Existing public datasets typically provide only graph-level binary labels [43], [66] or coarse attack descriptions [32]. The lack of fine-grained TTP annotations and limited coverage of TTP types make them unsuitable for evaluating TTP recognition. To fill this gap, we built and released KELLECT4APT, the first kernel-level APT dataset with diverse TTP instances and detailed annotations.

We constructed the dataset using the kernel-level provenance collection tool KELLECT [36] and the Atomic Red Team framework developed by Red Canary, which provides modular TTP scripts and corresponding attack reports. During data collection, each script was executed in Windows virtual machines (VMware[4]) via PowerShell, while KELLECT continuously captured provenance logs. To approximate real-world conditions, we also introduced benign background activities (e.g., web browsing, file downloads). To protect privacy, we ensured that 1) all background activities used only synthetic content; 2) system identifiers were anonymized or replaced with generic placeholders; and 3) virtual environments were reverted to a clean snapshot after each TTP execution to eliminate residual state contamination. These precautions prevent sensitive data from being included in the released dataset. We further refined the dataset by discarding duplicate, failed, or incomplete executions, resulting in 473 complete system traces of TTP activities spanning 9 APT tactics, 26 techniques, and 45 sub-techniques. Table IV summarizes the data distribution. Using KELLECT4APT, we then constructed the following two evaluation subsets.

*NOI Detection Subset:* This subset is used to evaluate the NOI detection. To establish ground truth, we adopted the annotation strategy described in [23]. For each TTP execution, IoCs were extracted from the Atomic Red Team's attack reports and mapped to the corresponding provenance graph to identify matched nodes. All process nodes within 2 hops of the matched nodes were subsequently labeled as NOIs, resulting in a total of 647 process NOIs. In addition, we deployed KELLECT on multiple hosts to collect benign evaluation data. Consistent with prior studies [23], [24], we assumed that no malicious behaviors occurred during this period. This process yielded 23.9 GB of audit logs with 12,743 process nodes.

[4]https://www.vmware.com/

TABLE IV
TTP Distributions of Dataset KELLECT4APT

| APT Tactic | APT (Sub-)Technique | # of APT Procedures |
|---|---|---|
| Initial Access | T1566.001 | 2 |
| Credential Access | T1558.001,T1552.004,T1558.004,T1003.001, T1003.002,T1003.003,T1552.001,T1552.002, T1555.003,T1552.006,T1110.003,T1003.006, T1556.002 | 92 |
| Defense Evasion | T1562.006,T1562.004,T1562.001,T1218.007, T1218.005,T1218.011,T1218.004,T1070.004, T1548.002,T1222.001,T1562.002,T1070.005, T1218.010,T1556.002,T1134.004,T1218.001, T1036.003 | 211 |
| Persistence | T1547.001,T1137.006,T1547.004,T1543.003, T1556.002,T1053.005 | 64 |
| Privilege Escalation | T1547.001,T1548.002,T1547.004,T1543.003, T1134.004,T1053.005 | 85 |
| Discovery | T1087.002,T1069.002,T1614.001,T1069.001 | 46 |
| Execution | T1204.002,T1569.002,T1059.001,T1059.003, T1053.005 | 60 |
| Exfiltration | T1048.003 | 9 |
| Impact | T1491.001 | 2 |

TABLE V
Dataset Distribution of NOI Detection

| Node Distribution | | # of Process Nodes |
|---|---|---|
| Train | Benign | 8,920 |
| Test | Malicious | 647 |
| | Benign | 3,823 |

*TTP Recognition Subset:* Leveraging the modular design of Atomic Red Team scripts, where each script simulates an APT procedure aligned with a specific ATT&CK technique or sub-technique, we thus directly mapped the collected provenance graphs to their corresponding TTP labels. This automatic procedure removes subjective bias. To ensure a robust evaluation, we retained only technique labels supported by at least five distinct scripts (i.e., each label has five different samples). This process yielded 160 provenance graphs covering 7 tactics and 32 distinct sub-techniques/techniques. From these, we constructed two evaluation datasets: 1) ground-truth TTP subgraphs (denoted as $gTSG$), manually extracted by experts; and 2) sampled TTP subgraphs (denoted as $sTSG$), automatically generated using our NOI detection model and sampling algorithm (see Section IV-C). These subsets provide a rigorous basis for evaluating TTP recognition.

*2) Evaluation Strategies:* We define the following evaluation strategies for different tasks.

*NOI Detection:* The NOI detection model is evaluated on an unbalanced dataset with a benign-to-malicious node ratio of 6:1. Specifically, benign processes were segregated into a training set and a testing set at a ratio of 7:3. In addition, 647 annotated malicious nodes were incorporated into the testing set. The data distribution for NOI detection is detailed in Table V. The

evaluation metrics include Recall, False Positive Rate (FPR), Accuracy, Precision, F1-score, and ROC AUC score.

*TTP Recognition:* We evaluate TTP recognition performance under two experimental scenarios: $gTSGs$ are used to specifically evaluate the model's ability to identify TTPs in a noise-free setting, while $sTSGs$ provide an end-to-end performance evaluation of ZOOMER under realistic deployment conditions. For both settings, we report TTP recognition accuracy at three hierarchical levels:

- *Sub-technique Accuracy (Sub-tech. Acc.):* The proportion of $TSGs$ with correctly predicted sub-technique labels.
- *Technique Accuracy (Tech. Acc.):* The proportion of $TSGs$ whose predicted technique labels are correct.
- *Tactic Accuracy (Tac. Acc.):* The proportion of $TSGs$ whose predicted tactic labels are correct.

As the evaluation shifts from sub-techniques to tactics, the classification label space is narrowed (from 32 to 7), thereby simplifying the TTP recognition task but reducing the depth of analytical insight. We provide a comprehensive evaluation of ZOOMER across different recognition granularities. To ensure statistical reliability, all reported results are the average performance of 10 independent runs with varying random seeds.

*3) Experiment Environment:* All experiments were conducted on a server featuring Intel Xeon Gold 5128 CPUs, NVIDIA RTX 4090 GPUs, and 128 GB of RAM. The implementation of ZOOMER contains about 7,500 lines of Python codes. We utilized NetworkX[5] for provenance graph construction, and the NOI detection model was built with Scikit-Learn.[6] The Deep & Wide model and the subgraph sampling algorithm were developed using PyTorch and DGL.[7] The parameter settings of NOI detection and subgraph representation modules are detailed in Section V-B.

### B. Parameter Sensitivity Analysis

The first experiment explored the effects of different parameters on NOI detection and TTP recognition.

*1) Parameter Tuning for NOI Detection:* The NOI detection model involves two hyperparameters: the number of nearest neighbors $n$ to search and the anomaly threshold $\theta$. We conducted a sensitivity analysis to examine their impact on performance. Specifically, $n$ was varied from 5 to 20 and the AUC was used as the evaluation metric. As illustrated in Fig. 4(a), performance decreased steadily with larger values of $n$. Fixing $n$, we then varied $\theta$ between 7 and 15 and measured performance using the F1-score. As shown in Fig. 4(b), the F1-score peaked at $\theta = 12$. Based on these observations, we adopt $n = 5$ and $\theta = 12$ as the default configuration in all subsequent experiments.

*2) Parameter Tuning for Subgraph Representation:* This experiment tuned two hyperparameters of the deep component: the embedding dimension $d$ and the number of GAT layers $l$. The embedding dimension was varied from 64 to 256, and the number of layers from 1 to 4, while keeping others fixed.

[5]https://networkx.org/
[6]https://scikit-learn.org/.
[7]https://www.dgl.ai/

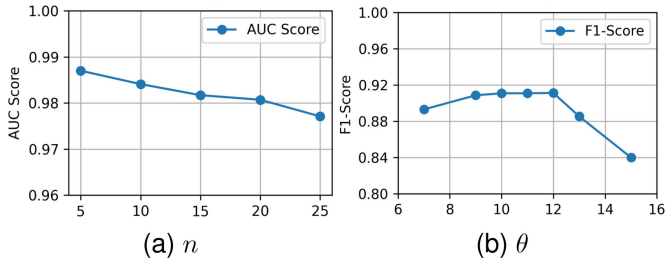


Fig. 4. The effect of neighbors $n$ and threshold $\theta$ on the NOI detection performance.

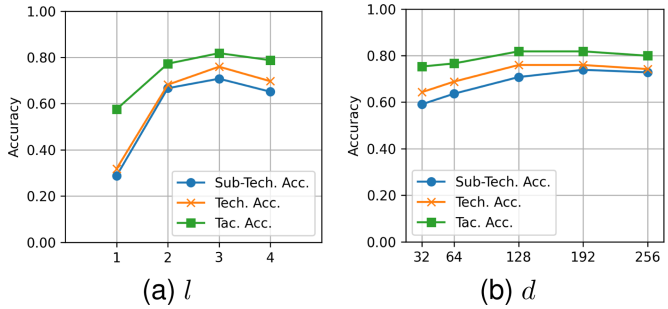


Fig. 5. The impact of parameters $d$ and $l$ on the TTP recognition performance.

According to the experiment results in Fig. 5(a), recognition performance improves as $d$ increases but saturates at $d = 128$. This phenomenon can be attributed to the fact that larger dimensions introduce noise and lead to performance degradation, while overly small dimensions fail to capture the behavioral semantics of TTP patterns. Fig. 5(b) shows a similar trend for $l$. Performance is poor at $l = 1$, suggesting that first-order neighbor aggregation is insufficient. Performance peaks at $l = 3$, indicating that incorporating up to third-order interactions adequately captures contextual information. Balancing accuracy and computational efficiency, we adopt $d = 128$ and $l = 3$ in subsequent experiments.

### C. Ablation Study

We conducted ablation experiments to evaluate the contribution of Deep & Wide model to multi-granular TTP recognition. Specifically, we compared three model variants:

1) *Wide:* It utilizes the wide model for TTP representation;
2) *Deep:* It utilizes the deep model for TTP representation;
3) *Deep & Wide:* It merges both deep and wide models for TTP representation.

All three variants rely on their respective embedded prototypes for TTP recognition. The results, summarized in Table VI, lead to three key observations. First, the joint Deep & Wide model consistently outperforms the individual models, achieving the highest TTP recognition accuracy across all granularities. Importantly, even with $sTSGs$ sampled from real-world provenance graphs, ZOOMER maintains robust performance, demonstrating its resilience under realistic deployment conditions. Second, the wide-only model performs about 16%

TABLE VI
IMPACT OF GRAPH REPRESENTATION MODELS ON TTP RECOGNITION ACCURACY

| Model | Groundtruth Subgraph ($gTSG$) | | | Sampled Subgraph ($sTSG$) | | |
|---|---|---|---|---|---|---|
| | Sub-tech. Acc. | Tech. Acc. | Tac. Acc. | Sub-tech. Acc. | Tech. Acc. | Tac. Acc. |
| Wide | 0.7208 | 0.7403 | 0.8117 | 0.5722 | 0.5910 | 0.6852 |
| Deep | 0.7590 | 0.7922 | 0.8377 | 0.6000 | 0.6533 | 0.7533 |
| Deep & Wide | 0.8851 | 0.8966 | 0.9425 | 0.8321 | 0.8596 | 0.9132 |

TABLE VII
COMPARISON OF NOI DETECTION WITH STATE-OF-THE-ART NODE-LEVEL APT DETECTION METHODS. BEST RESULTS ARE BOLDED. SECOND-BEST ARE UNDERLINED.

| NOI Detection Model | Recall | FPR | Accuracy | Precision | F1-score | AUC Score |
|---|---|---|---|---|---|---|
| THREATRACE | 0.7975 | **0.0152** | 0.8414 | **0.9942** | 0.8851 | 0.8912 |
| MAGIC | 0.9815 | 0.0856 | 0.9225 | 0.6135 | 0.7551 | **0.9936** |
| TREC | 0.9118 | 0.0674 | 0.9431 | 0.7552 | 0.8335 | 0.9537 |
| ZOOMER | **0.9845** | 0.0244 | **0.9765** | 0.8482 | **0.9113** | 0.9870 |

worse, suggesting that deep semantic representations of provenance graphs are crucial for enhancing generalization. Third, the deep-only model lags behind by 12%–23%, underscoring the value of incorporating prior knowledge and cross-feature interactions in the wide component. These features complement the deep embeddings by capturing recurrent patterns within the same TTP class, thereby boosting recognition performance. In summary, these findings validate the complementary strengths of the deep and wide models and the necessity of their integration for accurate TTP recognition.

### D. Comparative Experiment

This experiment compared the NOI detection and the TTP recognition models with the state-of-the-art baselines.

*1) NOI Detection Comparison:* We evaluated our NOI detection model in comparison with the following APT detection methods on the process NOI detection task.

1) *THREATRACE [23]:* It transforms the APT detection problem into a node type classification task. Specifically, it trains a hierarchy of node type classification sub-models based on the structural features, and detects malicious nodes as those misclassified by sub-models.
2) *MAGIC [24]:* It pre-trains the GNN model to capture node contextual behaviors via masked self-supervised representation learning, and identifies malicious nodes through the distance-based anomaly detection.
3) *TREC [67]:* It employs a heterogeneous graph attention network to learn node embeddings by traversing predefined meta-paths. Then, it pinpoints malicious nodes using the distance-based anomaly detection.

We employed the open-source implementations of THREATRACE and MAGIC, training them from scratch on our datasets and tuning their hyperparameters following the optimization details specified in their respective papers. For TREC, as our prior work, we directly applied its pre-trained model to the evaluation datasets. From the experiment results in Table VII, it can be concluded that our method achieves optimal performance in terms of Recall, Accuracy, and F1-score. Both THREATRACE and TREC yield lower F1-scores, reflecting their inability to balance Recall and FPR. This performance gap can be attributed to their narrow focus on node structural semantics while neglecting richer attribute semantics. Similarly, MAGIC underperforms across Recall and FPR, further confirming that jointly modeling structural and semantic features yields a more informative representation of process nodes for NOI detection.

*2) TTP Recognition Comparison:* We benchmarked ZOOMER against five representative baselines, categorized into heuristic rule-based and graph learning-based approaches. We present our evaluations in two distinct parts for clarity.

*Comparison with Rule-based Methods:* We first compared ZOOMER with two heuristic rule-based approaches:

1) *Holmes [28]:* It designs a set of TTP specifications to map low-level events to high-level TTP activities within the Kill Chain framework. Moreover, it also leverages the information flow analysis to correlate ancestor processes to infer tactic labels of suspicious nodes.
2) *APTShield [6]:* It introduces an APT detection framework based on the ATT&CK matrix. The framework defines suspicious labels for nodes, customizes label propagation rules, and ultimately recognizes ATT&CK tactics through aggregation of target node labels.

Both heuristic baselines are restricted to tactic-level recognition and operate at the node level. For a fair comparison, we evaluated their subgraph-level APT tactic recognition performance via majority voting over node-level predictions. Consistent with their original designs, Holmes was evaluated under the Kill Chain model, while APTShield was assessed with respect to the ATT&CK framework. Since Holmes is not publicly available, we re-implemented it following the original specification [28]. Our prior work APTShield, initially developed for Linux, was also re-implemented to a Windows-compatible version. The implementation details are provided in Appendix D, available online.

As shown in Table VIII, ZOOMER significantly outperforms Holmes and APTShield, with margins of ∼17% and ∼30%, respectively. The inferior performance of heuristic methods

TABLE VIII
TACTIC RECOGNITION ACCURACY COMPARISON WITH HEURISTIC RULE-BASED BASELINES. BEST RESULTS ARE BOLDED.

| Model | APT Tactic Framework | |
|---|---|---|
| | Kill Chain | ATT&CK |
| APTShield | - | 0.7747 |
| Holmes | 0.5352 | - |
| ZOOMER | **0.8310** | **0.9425** |

TABLE IX
TTP RECOGNITION ACCURACY COMPARISON WITH GRAPH LEARNING-BASED BASELINES. BEST RESULTS ARE BOLDED. SECOND-BEST ARE UNDERLINED.

| Model | Sub-tech. Acc. | Tech. Acc. | Tac. Acc. |
|---|---|---|---|
| NeuroMatch | 0.3250 | 0.3688 | 0.4352 |
| GHunter | 0.4363 | 0.4875 | 0.5780 |
| ProvG-Searcher | <u>0.5813</u> | <u>0.6188</u> | <u>0.6869</u> |
| ZOOMER | **0.8321** | **0.8596** | **0.9132** |

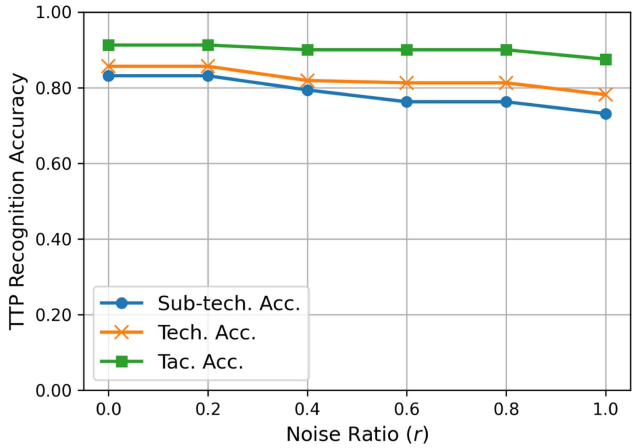


Fig. 6. Sensitivity of TTP recognition to $TSG$ variations.

stems from two main limitations. First, their reliance on expert knowledge alone fails to capture the full diversity and complexity of multi-step TTP patterns, leading to reduced recognition accuracy. Second, their dependence on rigid black-white lists often results in misidentifications among TTPs with similar signatures. In contrast, ZOOMER overcomes these issues while achieving higher recognition accuracy and stronger adaptability across diverse tactic frameworks.

*Comparison with Learning-based Methods:* We next evaluated ZOOMER against three learning-based approaches:

1) *NeuroMatch [52]:* It introduces an accurate and efficient neural network-based inexact subgraph matching method that assumes the partial order of subgraph relationships is inherent aligned with order embedding and can be exploited to predict subgraph matches.
2) *GHunter [51]:* It abstracts nodes with similar semantics across compared subgraphs into uniform types, and employs Graph Isomorphism Network (GIN) [68] to generate graph representations via node and edge embeddings. Then, subgraph correlation prediction, inspired by NeuroMatch, is utilized to identify known attacks.
3) *ProvG-Searcher [7]:* It uses GNN to embed subgraphs centered on process nodes, integrating node, edge, and time information. Like GHunter, it predicts subgraph relationships based on the order embedding technique.

The intuition behind this comparison is that the baselines share a similar workflow with ZOOMER. Specifically, the baseline leverages attack graphs extracted from CTI reports as templates to identify potential threats in provenance graphs, which parallels ZOOMER's approach of using TTP subgraphs as templates to detect corresponding TTP patterns. In our setup, the baselines employ manually extracted $gTSGs$ as query graphs and perform subgraph matching on sampled $sTSGs$ for TTP recognition, with the matched $gTSGs$'s label returned as the recognition result. This configuration is consistent with their original use cases. Moreover, we enforced that target graphs are strictly larger than $gTSGs$, thereby preserving the intended subgraph entailment constraints described in their original work. NeuroMatch and ProvG-Searcher were evaluated using their open-source implementations, while GHunter was re-implemented by extending NeuroMatch's code with the reinforcement mechanisms described in the original paper [51]. Following the setup in [52], training datasets for all learning-based baselines were generated from benign provenance graphs.

As demonstrated in Table IX, ZOOMER consistently outperforms the baselines by over 25% across all TTP recognition granularities. The limitations of the baselines can be attributed to the following factors. First, they restrict the size of query and target graphs, which constrains the semantic expressiveness of different $TSGs$. Second, their node abstraction in the provenance graph weakens the deep semantic representation among $TSGs$, leading to multiple $TSGs$ with similar representations and thus degrading accuracy. Third, they convert provenance graphs to undirected representations, preventing deep learning models from capturing meaningful information flow directions. Fourth, ProvG-Searcher leverages the behavior semantic preservation scheme to deduplicate repeated behaviors, potentially eliminating specificity between $TSGs$. In contrast, ZOOMER adopts a Deep & Wide model that jointly captures structural and semantic node features while leveraging prior knowledge to better distinguish subtle differences among TTP patterns. Furthermore, the prototypical network aligns intra-class samples by pulling their representations closer, while simultaneously maximizing inter-class separation. This design enhances the performance under few-shot learning conditions.

### E. Sensitivity Analysis

We evaluated the sensitivity of TTP recognition to perturbations in sampled $TSGs$. To simulate such perturbations, we injected previously unseen but benign interactions that bypass the heuristic subgraph sampling rules, thereby introducing controlled noise into each $TSG$. A noise ratio $r \in [0, 1]$ is defined to quantify the proportion of injected nodes.

Experimental results in Fig. 6 reveal that ZOOMER preserves stable performance across different TTP recognition

TABLE X
OVERHEAD OF ZOOMER'S KEY COMPONENTS

| Component | Phase | Time Cost (s / 1k nodes) | Memory Cost (MB / 1k nodes) |
|---|---|---|---|
| NOI Detection | Feature Extraction | 0.71 | 1 |
| | NOI Detection | 0.04 | 48 |
| Subgraph Sampling | Graph Traversal and Pruning | 2.25 | 3 |
| TTP Recognition | Wide Representation | 1.82 | 258 |
| | Deep Representation | 0.01 | 468 |
| | Recognition | 0.05 | 150 |

granularities as $r$ increases, indicating strong resilience to various semantic perturbations. Concretely, tactic-level recognition proves the most robust, with accuracy consistently above 87% regardless of noise ratio, partially due to the smaller label space reducing the task complexity. Technique-level recognition displays only a mild downward trend, while sub-technique recognition experiences the largest performance drop, yet the degradation is bounded at roughly 9%. In summary, these results confirm that ZOOMER is robust against $TSG$ precision loss. We attribute this resilience to the knowledge-driven feature engineering, which yields generalized TTP representations less susceptible to simple additive noise.

### F. Overhead Experiment

This experiment assesses the computational and memory overheads of ZOOMER's key modules, including the NOI detection, subgraph sampling, TTP subgraph representation and recognition. The results are summarized in Table X.

*1) Overhead of NOI Detection:* The overhead of NOI detection comprises node feature extraction and anomaly detection. As shown in Table X, extracting features for 1,000 nodes requires an average of 0.71 s and less than 1 MB of memory. Searching and calculating distance to the 5 nearest neighbors takes 0.05 s on average for 1,000 nodes, with a memory footprint around 48 MB, primarily for storing the baseline embeddings of the benign process nodes.

*2) Overhead of Subgraph Sampling:* The primary computational overhead in $TSG$ sampling arises from graph traversal and pruning strategies. Table X reveals that sampling a $TSG$ with 1,000 nodes costs on average 2.25 s and 3 MB of constant memory, showcasing its high efficiency. We further evaluated the sampling performance on provenance graphs ranging from 1,500 to 50,000 nodes, which reflects the typical graph scale in real-world scenarios. The results, presented in Fig. 7, indicate that the execution time generally lies between 1 s and 5 s, with heuristic pruning accounting for the dominant cost. At smaller scales, each additional 1,000 nodes in the provenance graph increases the runtime by roughly 0.5 s, reflecting the rising complexity of local structures handled during pruning. As the graphs become larger, the sampling time becomes more variable due to fluctuations in local structural patterns. Notably, the runtime peaks at around 5 s, suggesting an upper bound on the local structural complexity. Moreover, the size of sampled $TSG$ does not scale proportionally with the the original graph size, underscoring the robustness of our sampling algorithm against noisy neighbor nodes.

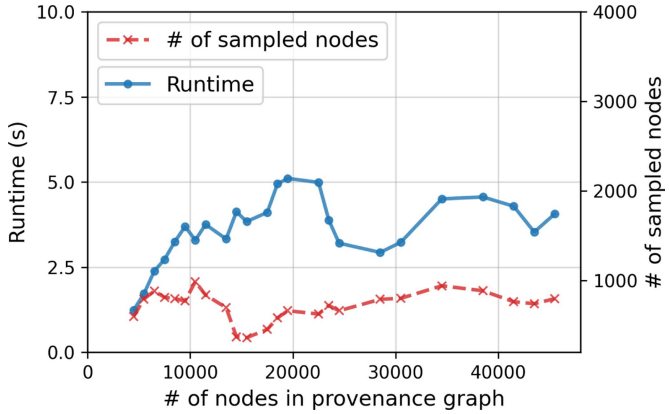


Fig. 7. Overhead analysis of subgraph sampling algorithm in provenance graphs with different scales.

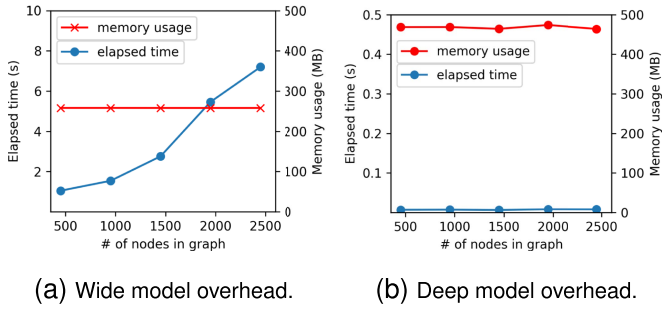


Fig. 8. Overhead analysis of graph representation across varying graph sizes.

*3) Overhead of TTP Recognition:* The TTP recognition pipeline comprises two main stages: $TSG$ representation and TTP prototype matching. Table X highlights that the average runtime of TTP recognition for a $TSG$ is less than 2 s, with a stable memory footprint under 1 GB. More precisely, Figs. 8(a) and (b) illustrate the effect of $TSG$ size on graph representation costs. The wide feature extraction stage incurs a constant memory overhead of 258 MB, primarily attributed to IoC knowledge base storage. While its runtime scales proportionally with the size of the $TSG$, the average size of a $TSG$ typically contains fewer than 1,000 nodes, ensuring a processing time as low as 2 s. The deep model loads a pretrained GAT with a fixed memory footprint of around 468 MB and shows negligible runtime variation across different $TSG$ sizes. The graph matching stage compares the $TSG$ embedding against various TTP prototypes, requiring about 150 MB of memory and only 0.05 s per $TSG$. Overall, the TTP recognition pipeline averages $\sim$2 s runtime and $\sim$876 MB memory usage, highlighting its efficiency and scalability. This performance allows ZOOMER to sustain a throughput of 1,200 alerts per hour, making it well-suited for handling large-scale malware incidents in enterprise environments [69].

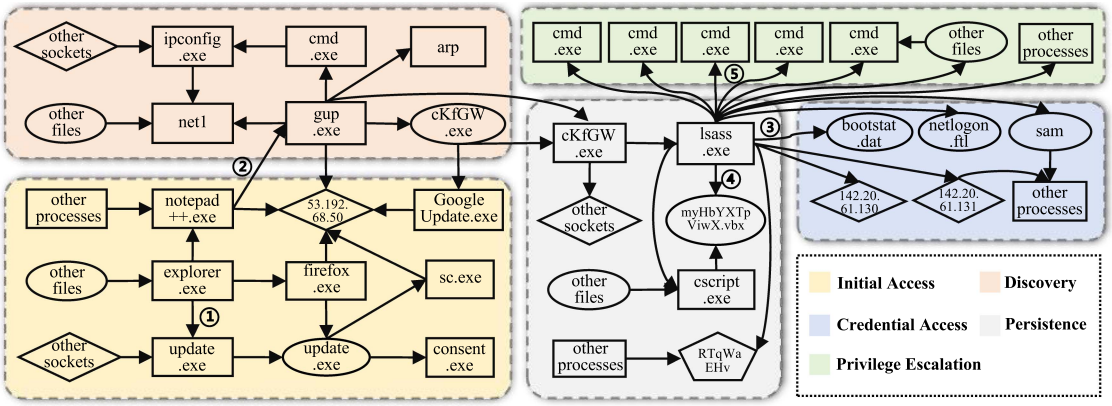


Fig. 9. TTP recognition case from the Darpa OpTC project: "Malicious Upgrade" attack initiated on Day 3. For clarity, only key entities involved in the attack are retained, while noisy entities generated during sampling are excluded. $TSGs$ associated with different tactics are distinguished using colored boxes, with circled numbers indicating the tactic transitions.

TABLE XI
TTP RECOGNITION ON THE DARPA OPTC DATASET

| APT Campaign | TTPs Used | Detected (Result) | Tech. Acc. | Tac. Acc. |
|---|---|---|---|---|
| Plain PowerShell Empire | T1059 | ✓ | 3/5 | 4/5 |
| | T1548 | ✓ | | |
| | T1069 | ✓ | | |
| | T1018 | × (T1069) | | |
| | T1070 | × (T1547) | | |
| Custom Powershell Empire | T1566 | ✓ | 4/5 | 4/5 |
| | T1059 | ✓ | | |
| | T1548 | ✓ | | |
| | T1083 | × (T1614) | | |
| | T1048 | ✓ | | |
| Malicious Upgrade | T1003 | ✓ | 3/5 | 5/5 |
| | T1059 | ✓ | | |
| | T1547 | ✓ | | |
| | T1018 | × (T1069) | | |
| | T1195 | × (T1566) | | |

### G. Real-World Attack Evaluation

To further assess the practical utility of ZOOMER in real-world scenarios, we conducted experiments on the DARPA OpTC dataset [70], which comprises three realistic multi-stage APT campaigns: "Plain PowerShell Empire", "Custom PowerShell Empire", and "Malicious Upgrade", all executed within an enterprise-scale environment comprising more than 1,000 Windows hosts. Given the lack of available TTP annotations, we engaged three experts to independently annotate ATT&CK tactics for each campaign based on official attack reports. Discrepancies in annotations were resolved through majority voting to achieve consensus.

The experimental results in Table XI reveal that ZOOMER attains accuracy exceeding 80% for APT tactic recognition. Mainstream techniques (e.g., T1059) are correctly classified, indicating that ZOOMER effectively captures their distinctive behavior characteristics. Misclassifications mainly occur with techniques such as T1083 and T1018, primarily due to the absence of their implementations in the Red Canary repository, limiting the availability of TTP prototypes. Notably, despite the lack of a prototype for T1195, its $TSG$ was mapped to the semantically related technique T1566 (both under the Initial Access tactic). By exploiting this semantic proximity, ZOOMER proves its effectiveness and robust generalization.

### H. Case Study

We used the third attack campaign "Malicious Upgrade" from DARPA OpTC as a case to illustrate how TTP recognition aids the APT investigation. As shown in Fig. 9, the campaign begins with a `Notepad++` update (① Initial Access), which downloads `update.exe` to stage a Meterpreter payload. Thereafter, the adversary invokes `cmd.exe` to enumerate the local system, network, and shared resources (② Discovery). To enhance its foothold, the implant migrates into the `lsass.exe` process, a common technique to evade detection. Using this substitute, the attacker proceeds to dump credentials from LSASS memory (③ Credential Access), reinforces persistence through script deployment and registry modification (④ Persistence), and finally creates a new administrator account for remote access (⑤ Privilege Escalation).

Fig. 9 illustrates the resulting TTP-annotated attack chain. Our evaluation revealed that while process NOI detection largely narrowed down the investigative scope, the presence of numerous adjacent interactions and unclear behavior boundaries still posed challenges for in-depth analysis. ZOOMER effectively addresses this by sampling each NOI-centric $TSG$ (colored subgraph in Fig. 9) and aligning them with correlated APT tactics, yielding a concise, well-bounded, and interpretable representation. This process directs analysts' attention to single-stage attack activities and provides fine-grained TTP insights, thereby reducing analytical complexity and accelerating incident response.

## VI. DISCUSSION AND LIMITATIONS

*Practicality:* Current research predominantly centers on the detection of isolated malicious activities [7], [23], [24], [49], while failing to provide an interpretable view of correlated attack stages. ZOOMER serves as a practical assistant for security analysts by streamlining multiple analysis tasks. First, for *false alarm reduction*, it automatically samples subgraphs around

alarm nodes and validates alerts by matching behavior patterns against known TTPs. Second, in *attack scenario reconstruction*, ZOOMER maps anomalous behaviors to corresponding TTPs, thereby enabling analysts to piece together complete APT attack chains. Third, for *threat hunting*, it uses TTP-driven query graphs to proactively search for known adversarial activities and strengthen defensive measures.

*Concept Drift:* Both the NOI detection and TTP recognition in ZOOMER are susceptible to concept drift. For NOI detection, the model relies on historical benign patterns and is thus prone to triggering false positives when new normal behaviors are encountered, a limitation also observed in prior work [24], [25], [42]. This issue can be alleviated through periodic retraining. For TTP recognition, drift occurs when novel TTP patterns emerge without corresponding prototypes. To address this, we adopt a hierarchical validation strategy: $TSGs$ are first matched against prototypes at the technique level, and unmatched samples are further evaluated at the broader tactic level. This two-stage validation improves adaptability to unseen TTPs while preserving recognition fidelity.

*Adversarial Robustness:* Adversarial attacks pose additional challenges, as sophisticated attackers may deliberately craft evasive behaviors to exploit detection boundaries. However, evading ZOOMER's detection is non-trivial in many cases, due to the following design choices. First, our NOI detection model leverages both semantic and structural features to identify process NOIs, providing greater robustness against adversarial perturbations compared to previous approaches that rely solely on structural features. Second, our TTP subgraph sampling algorithm heuristically prunes previously observed substructures, rendering naive addition-only perturbations ineffective [71]. Third, our TTP recognition model demonstrates resilience against imprecise TTP subgraphs (see Section V-E), which strengthens robustness against adversarial manipulations. Importantly, while ZOOMER cannot recognize entirely novel TTP patterns, it remains capable of detecting previously unseen attacks when their execution patterns align with known TTP prototypes (see Section V-G).

*Limitations:*ZOOMER relies on a diverse set of TTP subgraphs during the training phase to enhance detection capability. At present, KELLECT4APT covers 473 TTP instances from Red Canary's repository [35] on the Windows platform, which still covers only a subset of the full ATT&CK framework. Expanding this scope by incorporating additional TTP samples through automated generation tools or extracting TTP patterns from CTI reports is essential. Furthermore, extending the dataset to multiple platforms (e.g., Linux) will increase ZOOMER's generalization capacity.

## VII. Conclusion and Future Work

This paper presents ZOOMER, a novel framework for multi-granularity TTP recognition. By integrating an NOI detection model with a heuristic subgraph sampling algorithm, ZOOMER segments provenance graphs into $TSGs$ with precise boundaries, enabling both fine-grained attack analysis and accurate TTP recognition. For recognition, we design a Deep & Wide dual-tower architecture that unifies a deep model for generalized graph representation learning with a wide model for domain-informed feature memorization, producing comprehensive and discriminative TTP representations. To further enhance generalization under limited supervision, this architecture is embedded into a prototypical network, which supports few-shot TTP recognition. Extensive experiments demonstrate that ZOOMER achieves recognition accuracies of 88% at the sub-technique level and 94% at the tactic level, substantially outperforming existing baselines.

Our future work aims to extend KELLECT4APT by generating more TTP samples across different platforms, thereby strengthening ZOOMER's ability to recognize newly emerging attack patterns. Furthermore, we plan to incorporate adversarial training for graph-based detection models and integrate uncertainty quantification into TTP recognition, with the goal of improving resilience against adaptive adversaries.

## References


[1] S. T. H. Team, "Daggerfly: Apt actor targets telecoms company in africa," 2023. [Online]. Available: https://symantec-enterprise-blogs.security.com/blogs/threat-intelligence/apt-attacks-telecoms-africa-mgbot

[2] M. Corporation, "Mitre ATT&CK," 2015. [Online]. Available: https://attack.mitre.org

[3] P. Fang, P. Gao, C. Liu, E. Ayday, K. Jee, and T. Wang, "Back-propagating system dependency impact for attack investigation," *31st USENIX Secur. Symp.*, pp. 2461–2478, 2022.

[4] M. A. Inam et al., "SOK: History is a vast early warning system: Auditing the provenance of system intrusions," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 2620–2638.

[5] T. Zhu et al., "General, efficient, and real-time data compaction strategy for APT forensic analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 3312–3325, 2021.

[6] T. Zhu et al., "APTSHIELD: A stable, efficient and real-time APT detection system for linux hosts," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 5247–5264, Nov./Dec. 2023.

[7] E. Altinisik, F. Deniz, and H. T. Sencar, "ProVG-Searcher: A graph representation learning approach for efficient provenance graph search," in *Proc. 2023 ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2023, pp. 2247–2261.

[8] Y. Liu, X. Shu, Y. Sun, J. Jang, and P. Mittal, "Rapid: Real-time alert investigation with context-aware prioritization for efficient threat discovery," in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, Dec. 2022, pp. 827–840.

[9] W. U. Hassan et al., "NoDoze: Combatting threat alert fatigue with automated provenance triage," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, Feb. 2019.

[10] M. E. Aminanto, L. Zhu, T. Ban, R. Isawa, T. Takahashi, and D. Inoue, "Automated threat-alert screening for battling alert fatigue with temporal isolation forest," in *Proc. 17th Int. Conf. Privacy, Secur. Trust*, 2019, pp. 1–3.

[11] T. V. Ede et al., "DeepCase: Semi-supervised contextual analysis of security events," in *Proc. IEEE Symp. Secur. Privacy*, May 2022, pp. 522–539.

[12] T. Ban, N. Samuel, T. Takahashi, and D. Inoue, "Combat security alert fatigue with ai-assisted techniques," in *Proc. Cyber Secur. Experimentation Test Workshop*, Aug. 2021, pp. 9–16.

[13] C. Fu, Q. Li, K. Xu, and J. Wu, "Point cloud analysis for ml-based malicious traffic detection: Reducing majorities of false positive alarms," in *Proc. 2023 ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2023, pp. 1005–1019.

[14] Z. Yu, J. Wang, B. Tang, and L. Lu, "Tactics and techniques classification in cyber threat intelligence," *Comput. J.*, vol. 66, no. 8, pp. 1870–1881, 2023.

[15] X. Wang, X. Yang, X. Liang, X. Zhang, W. Zhang, and X. Gong, "Combating alert fatigue with alertpro: Context-aware alert prioritization using reinforcement learning for multi-step attack detection," *Comput. Secur.*, vol. 137, Feb. 2024, Art. no. 103583.

[16] A. Alsaheel et al., "Atlas: A sequence-based learning approach for attack investigation," *30th USENIX Secur. Symp.*, pp. 3005–3022, 2021.

[17] H. Ding, J. Zhai, Y. Nan, and S. Ma, "{AIRTAG}: Towards automated attack investigation by unsupervised learning with log texts," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 373–390.

[18] R. Patil et al., "E-Audit: Distinguishing and investigating suspicious events for APTs attack detection," *J. Syst. Archit.*, vol. 144, Nov. 2023, Art. no. 102988.

[19] N. Yan et al., "Deepro: Provenance-based apt campaigns detection via GNN," in *Proc. 2022 IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun.*, 2022, pp. 747–758.

[20] K. Pei et al., "Hercule: Attack story reconstruction via community discovery on correlated log graph," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, Dec. 2016, pp. 583–595.

[21] E. Hu, A. Fu, Z. Zhang, L. Zhang, Y. Guo, and Y. Liu, "Actracker: A fast and efficient attack investigation method based on event causality," in *Proc. 2021 IEEE Conf. Comput. Commun. Workshops*, May 2021, pp. 1–6.

[22] "Buran ransomware; the evolution of vegalocker," 2019. [Online]. Available: https://www.mcafee.com/blogs/other-blogs/mcafee-labs/buran-ransomware-the-evolution-of-vegalocker/

[23] S. Wang et al., "THREATRACE: Detecting and tracing host-based threats in node level through provenance graph learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 3972–3987, 2022.

[24] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, and M. Wen, "Magic: Detecting advanced persistent threats via masked graph representation learning," *33rd USENIX Secur. Symp.*, pp. 5197–5214, 2024.

[25] M. U. Rehman, H. Ahmadi, and W. U. Hassan, "Flash: A comprehensive approach to intrusion detection via provenance graph representation learning," in *Proc. 2024 IEEE Symp. Secur. Privacy*, 2024, pp. 3552–3570.

[26] T. Chen, "APT-KGL: An intelligent APT detection system based on threat knowledge and heterogeneous provenance graph learning," *IEEE Trans. Dependable Secure Comput.*, pp. 1–15, 2022.

[27] M. E. Ahmed, H. Kim, S. Camtepe, and S. Nepal, "PEELER: Profiling kernel-level events to detect ransomware," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2021, pp. 240–260.

[28] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, "HOLMES: Real-time apt detection through correlation of suspicious information flows," in *Proc. 2019 IEEE Symp. Secur. Privacy*, May 2019, pp. 1137–1152.

[29] C. Xiong et al., "CONAN: A practical real-time APT detection system with high accuracy and efficiency," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 551–565, Jan./Feb. 2022.

[30] "Sigma - Generic signature format for SIEM systems," 2022. [Online]. Available: https://github.com/SigmaHQ/sigma

[31] W. U. Hassan, A. Bates, and D. Marino, "Tactical provenance analysis for endpoint detection and response systems," in *Proc. IEEE Symp. Secur. Privacy*, May 2020, pp. 1172–1189.

[32] "Transparent computing engagement 3 data release," 2017. [Online]. Available: https://github.com/darpa-i2o/Transparent-Computing/blob/master/README-E3.md

[33] H.-T. Cheng et al., "Wide & deep learning for recommender systems," in *Proc. 1st Workshop Deep Learn. Recommender Syst.*, Sep. 2016, pp. 7–10.

[34] "Kellect4APT dataset," 2023. Available: https://www.kellect.org/#/kellect-4-aptdataset

[35] "Atomic red team," 2017. [Online]. Available: https://github.com/redcanaryco/atomic-red-team

[36] T. Chen, Q. Song, X. Qiu, T. Zhu, Z. Zhu, and M. Lv, "Kellect: A kernel-based efficient and lossless event log collector for windows security," *Comput. Secur.*, vol. 150, p. 104203, 2025.

[37] Z. Li et al., "Tags: Real-time intrusion detection with tag-propagation-based provenance graph alignment on streaming events," *CoRR*, 2024.

[38] Y. Liu et al., "Towards a timely causality analysis for enterprise security," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, vol. 24, 2018, p. 141.

[39] M. N. Hossain et al., "SLEUTH: Real-time attack scenario reconstruction from cots audit data," *26th USENIX Secur. Symp.*, pp. 487–504, 2017.

[40] R. Yang et al., "RATScope: Recording and reconstructing missing rat semantic behaviors for forensic analysis on windows," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1621–1638, May/Jun. 2020.

[41] S. Kok, A. Abdullah, and N. Jhanjhi, "Early detection of crypto-ransomware using pre-encryption detection algorithm," *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 34, no. 5, pp. 1984–1999, May 2022.

[42] Z. Cheng et al., "Kairos: Practical intrusion detection and investigation using whole-system provenance," *IEEE Symp. Secur. Privacy.*, pp. 3533–3551, 2024.

[43] X. Han et al., "Unicorn: Runtime provenance-based detector for advanced persistent threats," in *Proc. 2020 Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18.

[44] Y. Wu et al., "Paradise: Real-time, generalized, and distributed provenance-based intrusion detection," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 2, pp. 1624–1640, Mar./Apr. 2023.

[45] F. Yang et al., "{PROGRAPHER}: An anomaly detection system based on provenance graph embedding," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 4355–4372.

[46] Q. Wang et al., "You are what you do: Hunting stealthy malware via data provenance analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020.

[47] M. Balduzzi, V. Ciangaglini, and R. McArdle, "Targeted attacks detection with spunge," in *Proc. Privacy, Secur. Trust, 2013 11th Annu. Int. Conf.*, 2013, pp. 185–194.

[48] "Command and scripting interpreter," 2018. [Online]. Available: https://attack.mitre.org/techniques/T1059/

[49] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. Venkatakrishnan, "Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting," in *Proc. 2019 ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 1795–1812.

[50] R. Wei, L. Cai, L. Zhao, A. Yu, and D. Meng, "Deephunter: A graph neural network based approach for robust cyber threat hunting," in *Proc. Secur. Privacy Commun. Netw.*, 2021, pp. 3–24.

[51] Z. Cheng et al., "Ghunter: A fast subgraph matching method for threat hunting," in *Proc. 26th Int. Conf. Comput. Supported Cooperative Work Des.*, May 2023, pp. 1014–1019.

[52] Z. Lou et al., "Neural subgraph matching," 2020, *arXiv:2007.03092*.

[53] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 4080–4090.

[54] N. Rani, B. Saha, and S. K. Shukla, "A comprehensive survey of advanced persistent threat attribution: Taxonomy, methods, challenges and open research problems," *J. Inf. Secur. Appl.*, vol. 92, p. 104076, 2025.

[55] V. Sachidananda, R. Patil, A. Sachdeva, K.-Y. Lam, and L. Yang, "APTer: Towards the investigation of APT attribution," in *Proc. 2023 IEEE Conf. Dependable Secure Comput.*, 2023, pp. 1–10.

[56] A. Goyal, G. Wang, and A. Bates, "R-CAID: Embedding root cause analysis within provenance-based intrusion detection," in *Proc. 2024 IEEE Symp. Secur. Privacy*, 2024, pp. 3515–3532.

[57] Z. Zhang, P. Qi, and W. Wang, "Dynamic malware analysis with feature engineering and feature learning," in *Proc. AAAI Conf. Artif. Intell.* Apr. 2020, vol. 34, no. 01, pp. 1210–1217.

[58] L. E. Peterson, "K-nearest neighbor," *Scholarpedia*, vol. 4, no. 2, 2009, Art. no. 1883.

[59] J. Zeng, Z. L. Chua, Y. Chen, K. Ji, Z. Liang, and J. Mao, "Watson: Abstracting behaviors from audit logs via aggregation of contextual semantics," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021.

[60] A. Aly, S. Iqbal, A. Youssef, and E. Mansour, "MEGR-APT: A memory-efficient apt hunting system based on attack representation learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 5257–5271, 2024.

[61] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Li′o, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018.

[62] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th Int. Conf. Mach. Learn.*, 2010, pp. 807–814.

[63] Z. Li, Z. Qin, and P. Shen, "Intrusion detection via wide and deep model," in *Proc. Int. Conf. Artif. Neural Netw.*, 2019, pp. 717–730.

[64] J. Dougherty, R. Kohavi, and M. Sahami, "Supervised and unsupervised discretization of continuous features," in *Machine Learning Proceedings*. Amsterdam, The Netherlands: Elsevier, 1995, pp. 194–202.

[65] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. Probability*, 1967, pp. 281–298.

[66] "Streamspot data," 2014. [Online]. Available: https://github.com/sbustreamspot/sbustreamspot-data?tab=readme-ov-file

[67] M. Lv et al., "Trec: Apt tactic/technique recognition via few-shot provenance subgraph learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 139–152.

[68] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *Proc. Int. Conf. Learn. Representations*, 2019.

[69] "Top cybersecurity statistics for 2024," 2023. [Online]. Available: https://www.cobalt.io/blog/cybersecurity-statistics-2024

[70] "Operationally transparent cyber (OPTC) data release," 2020. [Online]. Available: https://github.com/FiveDirections/OpTC-data
[71] A. Goyal, X. Han, G. Wang, and A. Bates, "Sometimes, you aren't what you do: Mimicry attacks against provenance graph host intrusion detection systems," in *Proc. 30th Netw. Distrib. Syst. Secur. Symp.*, 2023.

**Xuebo Qiu** received the bachelor's degree from Wenzhou University, Wenzhou, China, in 2021. He is currently workjng toward the PhD degree with the Zhejiang University of Technology, Hangzhou, China. His research interests include system security and data mining.

**Mingqi Lv** received the PhD degree in computer science from Zhejiang University, China, in 2012. He is currently a full professor with the College of Geoinformatics, Zhejiang University of Technology, Huzhou, China. His research interests include spatial-temporal data mining and data-driven cyber security.

**Tieming Chen** received the PhD degree in computer software and theory from Beihang University, Beijing, China, in 2011. He is currently a full professor with the College of Geoinformatics, Zhejiang University of Technology, Huzhou, China. His research interests include cyberspace security and intelligence security.

**Tiantian Zhu** (Member, IEEE) received the PhD degree in computer science from Zhejiang university, Hangzhou, China, in 2019. He is currently an associate professor with the college of computer science and technology, Zhejiang University of Technology, China. His research interests include system security and artificial intelligence.

**Qijie Song** received the PhD degree in computer science from the Zhejiang University of Technology, Hangzhou, China, in 2025. He is currently a post-doctoral researcher with the college of geoinfomatics, Zhejiang University of Technology. His research interests include system security and spatiotemporal intelligence.

**Zhiling Zhu** received the PhD degree in computer science from the Zhejiang University of Technology, Hangzhou, China, in 2025. He is currently a post-doctoral researcher with the College of Geoinfomatics, Zhejiang University of Technology. His research interests include software engineering, open-source software, and automated CI/CD.